Probabilistic System Identification and State Estimation via Normalizing Flows

genehmigte Dissertation
zur Erlangung des akademischen Grades der Doktor-Ingenieurin (Dr.-Ing.).
an der
Fakultät für Angewandte Informatik
der
Universität Augsburg

vorgelegt von
Yi Zhang
aus
Tangshan, China

2026

Dekan: Prof. Dr. Jörg Hähner
Doktorvater: Prof. Dr.-Ing. Lars Mikelsons

I dedicate this thesis to my family.

Contents

Acronyms **i**

List of Figures **iv**

List of Tables **vii**

I Introduction **1**

1 Motivation 1

2 Background and Preliminaries 6

2.1 Notations 6

2.2 Bayesian Inference 8

2.2.1 Bayes' rule 8

2.2.2 Markov Chain Monte Carlo 9

2.2.3 Variational Inference 11

2.3 Deep Probabilistic Models 13

2.3.1 Deep Gaussian Process 13

2.3.2 Bayesian Neural Networks 15

2.3.3 Variational Autoencoder 16

2.4 Normalizing Flows 18

2.4.1 Basics 18

2.4.2 Coupling Flows 21

2.4.3 Continuous Flows 24

2.4.4 Conditional Normalizing Flows 26

2.5 From Inference to Control 27

2.5.1 Bayesian System Identification 27

2.5.2 Bayesian State Estimation 27

2.5.3 Model-free Control . . . 27
2.5.4 Model Predictive Control . . . 27
3 Literature Review . . . 28
3.1 Deep Probabilistic Models for Bayesian Inference . . . 28
3.2 Conditional Normalizing Flows . . . 28
3.3 Deep Probabilistic Models for System Identification and State Estimation . . . 28
3.4 Data-driven Control based on Deep Probabilistic Models . . . 28
4 Objectives and Contributions . . . 29
5 Outline . . . 30

II Methodologies **31**
1 Parameter Identification . . . 31
2 Dynamics Identification . . . 32
3 State Estimation . . . 33
4 Data-driven Control . . . 34

III Experiments and Results **35**
1 Overview . . . 35
2 Parameter Identification . . . 35
3 Dynamics Identification . . . 35
4 State Estimation . . . 35
5 Data-driven Control . . . 35

IV Discussion and Conclusion **37**

V Abstract **39**

VI Zusammenfassung **41**

Bibliography **42**

Own Contributions and Publications **43**
Own share in data acquisition and data analysis . . . 43
Own Publications . . . 43

Curriculum Vitae **52**

Appendix **55**

Acknowledgement **57**

Eidesstattliche Versicherung (Affidavit) in German **59**

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel **61**

Acronyms

DGM Deep Generative Model

VAE Variational Autoencoder

NF Normalizing Flow

BNN Bayesian Neural Network

RNN Recurrent Neural Network

NARX Nonlinear Autoregressive eXogenous

MPC Model Predictive Control

DGP Deep Gaussian Process

PDF Probability Density Function

CDF Cumulative Distribution Function

MCMC Markov Chain Monte Carlo

MH Metropolis-Hastings

MALA Metropolis-adjusted Langevin algorithm

HMC Hamiltonian Monte Carlo

ELBO Evidence Lower Bound

KL Kullback-Leibler

VI Variational Inference

GPR Gaussian process regression

SE squared exponential

RBF Radial Basis Function

DGP Deep Gaussian Process

SGD Stochastic Gradient Descent

CAVI Coordinate Ascent Variational Inference

BbB Bayes by Backprop

CNN Convolutional Neural Network

RNN Recurrent Neural Network

ODE ordinary differential equation

SDE stochastic differential equation

NICE Non-linear Independent Component Estimation

RealNVP Real-Valued Non-Volume Preserving

MAF Masked Autoregressive Flow

i.i.d. independent and identically distributed

INN Invertible Neural Network

cINN conditional Invertible Neural Network

HINT Hierarchical Invertible Neural Transport

MPC Model Predictive Control

List of Figures

I.1 Variational Autoencoder Architecture 17
I.2 Coupling Flow Architecture 22

List of Tables

Chapter I

Introduction

1 Motivation

In this seemingly certain world, uncertainty lingers like a shadow, yet from the uncertain, we often find certainty.

The paradoxical relationship between certainty and uncertainty manifests across disciplines, demonstrating how truth often arises from navigating ambiguity. Quantum mechanics redefines predictability under randomness in the subatomic world, where Newtonian deterministic physics collapses. This uncertainty led to groundbreaking laws (e.g. Heisenberg's Uncertainty Principle), which now underpin modern technologies like quantum computing and all forms of spectroscopy. An example of medical diagnosis is that a patient presents vague symptoms, such as fatigue or pain, which could indicate anything from stress to cancer. Doctors converge on the cause through iterative hypothesis-testing approaches. Climate science also epitomizes how we manage uncertainty. Climate models incorporate a wide range of sources of variability and uncertainty. A common way to address it is to combine the results of a collection of different simulation models. By running multi-model ensembles, scientists can identify consistent patterns across the uncertainty.

From quantum randomness to medical diagnosis to climate models, uncertainty does not destroy the deterministic systems, but provides us accessibility to identify the systems. This perspective supports the use of ***probabilistic inference*** methods, which offer a rigorous framework for representing uncertainty and reasoning from uncertainty. In statistics, modeling uncertainty of variables from given data samples, using a probability distribution rather than just a point estimate is known as inference. The estimated distribution facilitates uncertainty quantification and provides a measure of reliability of the results, making it particularly well-suited for real-world applications.

The mathematical foundation of probabilistic inference is Bayes' rule: beliefs about unknown variables (the prior) are updated using observed data (likelihood), yielding the posterior distribution. A simplified formulation is:

$$\text{posterior} = \text{prior} \times \text{likelihood} \,/\, \text{total evidence (or marginal likelihood)}. \tag{I.1}$$

For example, we can illustrate Bayes' rule using the medical diagnosis scenario: the objective is to obtain the probability of a disease given a positive test result:

$$P(\text{Disease}|\text{Positive}) = \frac{P(\text{Positive}|\text{Disease}) \times P(\text{Disease})}{P(\text{Positive})},$$
$$P(\text{Positive}) = P(\text{Positive}|\text{Disease}) \times P(\text{Disease}) + P(\text{Positive}|\overline{\text{Disease}}) \times P(\overline{\text{Disease}}),$$

where $\overline{\text{Disease}}$ denotes "No Disease". In the Bayesian context, probabilistic inference is the process of obtaining the posterior distribution of target variables from observed data. This perspective extends naturally to modern machine learning, where predictions, structures, and policies can all be understood as random variables and probability distributions (Murphy, 2022).

Especially in modern Deep Generative Models (DGMs), probabilistic inference plays an essential role (Murphy, 2023). Through introducing random variables to represent complex data in latent space, DGMs enable data synthesis from latent variables (generation) based on the recognition of underlying latent representation from data (inference). However, directly inferring the posterior distribution over the latent variables using Bayes' rule I.3 is generally intractable, since all possible values of the latent variables are rarely obtainable in complex and high-dimensional models. Addressing this challenge is essential to fully unfold the potential of DGMs. Variational Autoencoders (VAEs) (Kingma and Welling, 2019) tackle this problem by introducing an inference network as a tractable approximation to the posterior. Normalizing Flows (NFs) (Rezende and Mohamed, 2016) construct bijective transformations between observed data and latent variables, achieving exact computation of posteriors. Diffusion models (Ho et al., 2020), in contrast, sidestep calculating the posterior distribution over latent variables by designing it as a simple Gaussian, and instead frame both inference and generation as an iterative denoising process from noise through learned transformations.

To sum up, DGMs are inherently probabilistic; through probabilistic modeling and inference, they are powerful in extracting latent information and representing uncertainty in complex data. These capabilities are proven in numerous applications in images, text, and speech. The same principles extend naturally to scientific and engineering domains. One prominent example is: how can DGMs empower identification and modeling of complex, stochastic, nonlinear dynamical systems? In control theory, the process of building mathematical models of dynamical systems from observed input-output data is called ***system identification***. Interpreting system identification in probabilistic terms, dynamics identification is inferring the posterior of the next states based on the previous states and system inputs, while parameter identification is estimating the posterior over system parameters given observations.

Classic methods for system identification rely on linearized models or pre-specified nonlinear structures. Model parameters are then estimated from observed data using estimation techniques such as the least squares method and Bayesian estimation. Subsequently, a validation process is employed to assess and select the most appropriate model structure. Although these methods offer strong physical interpretability and are often data-efficient, they exhibit several limitations: parameter estimation can be computationally inefficient with large amounts of data; results are sensitive to the choice of model structures and dependent on expert knowledge; uncertainty is difficult to handle and quantify; and they are generally inapplicable to partially observable systems and high-dimensional and raw data.

Despite the demand for uncertainty quantification, deep learning methods are suitable for system identification thanks to their strength in representing information from complex data, flexibility of function approximation, scalability in high-dimensional data, and compatibility with optimization techniques. Taking uncertainty quantification into account, a simple yet effective technique is Gaussian Process Regression (Särkkä, 2019), which involves assigning a Gaussian process prior to the latent regressor and performing statistical conditioning based on the available observations. This method has gained widespread adoption due to its usability and adaptability. Nonetheless, its significant computational and memory requirements limit its feasibility when dealing with large datasets. Another straightforward approach to combine the advantages of deep learning methods and the possibility of uncertainty modeling is the Bayesian Neural Network (BNN) (Goan and Fookes, 2020). Unlike standard neural networks that produce point estimates, BNNs assign probability distributions over the network parameters, thereby inducing distributions for outputs. However, BNNs reveal expressiveness limitations because the approximate inference may not reliably capture true posterior behavior (Foong et al., 2020). Therefore, the adaptation and application of DGMs are crucial for achieving rich representational capacity, flexible modeling frameworks, and the ability to effectively handle large-scale data.

Different DGMs have been investigated in system identification applications. VAEs are flexible to cooperate with time series modeling architectures such as Recurrent Neural Network (RNN) (Girin et al., 2021) and Nonlinear Autoregressive eXogenous (NARX) (Paniagua and López, 2024), which is beneficial in uncovering the time-dependent features from data. However, VAEs suffers from a well-known problem of shrinkage of variance (Asperti, 2020) and tends to converge to a degenerate local optimum known as posterior collapse (Wang et al., 2023). Unlike VAEs, NFs are restricted in their architectural choices due to the requirement of invertibility, which limits their direct applicability to time series modeling. On the other side, NFs provide exact and expressive density estimation capabilities, qualifying them for capturing uncertainty in data and enhancing the representational

capacity of probabilistic forecasts. Diffusion models release the architectural constraints and manifest the strength in generating high-quality and coherent predictions. Nevertheless, their iterative data generation procedure incurs substantial computational overhead. Balancing expressiveness, computational efficiency, and sample quality, ***normalizing flows*** are chosen in this work as the generative backbone of probabilistic system identification.

Based on the results of system identification, ***state estimation*** is the process of inferring the value of hidden, time-varying state variables of a dynamical system from a history of noisy and potentially partial observations and known control inputs. Because of the noise in the sensors (measurement noise) and unmodeled disturbances in the system itself (process noise), state estimation is naturally probabilistic. Thus, estimating the target states is computing the posterior distribution of target states. The traditional method for state estimation is the Luenberger State Observer, which assumes the systems are deterministic and both the system dynamics and measurement models are linear and time-invariant. Taking stochasticity into account, Kalman Filter is the most widely applied method. Beyond system linearity, a Kalman filter assumes the process and measurement noises are additive Gaussian noise with zero mean. In order to handle nonlinear systems, Extended Kalman Filter linearizes the nonlinear functions around the state estimate. However, the linearization introduces errors, leading to wrong estimation or even divergence. An improved modification is the Unscented Kalman Filter, which carefully selects the means and variances and propagates them through the nonlinear functions. However, it is computationally inefficient and still only compatible with Gaussian noises.

Through relaxing the need for precise modeling of dynamical systems and avoiding restrictive assumptions about uncertainty—while preserving expressiveness, modeling flexibility, and scalability to large datasets—the chosen architecture for probabilistic system identification, NF, also provides a foundation for accurate, efficient and reliable probabilistic state estimation. From a data-centric perspective, parameter identification involves mapping time series to discrete data points; dynamics identification operates either between data points or across entire time series; and state estimation entails inferring data points or reconstructing time series from temporal data. In order to address these tasks, it is necessary to extend normalizing flows, given the limited research currently available in this area. Bridging this gap not only broadens the applicability of NFs but also contributes to advancing probabilistic inference techniques for complex temporal systems.

Having explored system identification and state estimation through probabilistic inference, we can advance toward data-driven control by leveraging data as a central source of insight into system behavior, enabling more adaptive, scalable, and robust control strategies. This is especially advantageous in situations where analytical models are incomplete, uncertain, or impractical to derive. One unmediated application is model-free control, which obtains control inputs directly from observations and reference trajectories. This

enables fast path planning without requiring an explicit system model. Furthermore, for more reliable and interpretable control, a preferable approach is to employ the identified dynamics within a Model Predictive Control (MPC) framework. This integration of learned models with numerical solvers and optimization routines allows for anticipatory decision-making based on predicted system behavior. Notably, in cases where simulation models are complex and the Jacobian of dynamics is analytically intractable, learning-based representations offer differentiable approximations that are inherently compatible with gradient-based solvers and optimizers. This compatibility facilitates efficient and scalable MPC implementations. Besides, the probabilistic nature of the extracted models enables uncertainty-aware control strategies, enhancing robustness and safety in dynamic and uncertain environments.

In summary, this work is motivated by three key considerations:

- Probabilistic inference provides a principled framework for modeling uncertainty and extracting knowledge from uncertainty, enabling system identification and state estimation based solely on observations. This is particularly valuable in scenarios where first-principles models are unavailable, incomplete, or prohibitively complex.
- System identification and state estimation are required to be expressive for representing complex models, capable of modeling uncertainty, and scalable to large data sets. Deep Generative Models (DGMs), in particular Normalizing Flows (NFs), fulfill these criteria. However, existing architectures must be further developed to adapt to sequential data more flexibly.
- The results of probabilistic system identification can be integrated in Model Predictive Control (MPC), paving the way for interpretable, reliable, and robust control strategies. This fusion of learned models with optimization-based control frameworks holds promise for advancing autonomous decision-making in complex, uncertain environments.

2 Background and Preliminaries

This section introduces the necessary foundational concepts and methodologies for understanding the rest of this work. Before the main contents, we explain the unified notations applied in this whole work. We first review classical Bayesian inference methods. Then we introduce deep probabilistic models, including Deep Gaussian Process (DGP), Bayesian Neural Network (BNN), and Variational Autoencoder (VAE). Next, we present the core methods of this work, Normalizing Flows (NFs), a powerful class of generative models for flexible density estimation. Finally, we bridge probabilistic inference to control methods, covering system identification, state estimation, and model predictive control.

2.1 Notations

To maintain clarity throughout this document, we adopt the following notation:

Basic symbols

- Scalars are denoted by lowercase letters, e.g., x.
- Vectors are denoted by bold lowercase letters, e.g., $\boldsymbol{x}$.
- Matrices are denoted by bold uppercase letters, e.g., $\boldsymbol{X}$.
- Sets or collections of mixed or unspecified data types are denoted by calligraphic letters, e.g., $\mathcal{D}$.

Random variables

- Multiple dimensional random variables are vectors, e.g., $\boldsymbol{x}$, which consist of single dimensional random variables, e.g., $[x_1, \ldots, x_N]^T$.
- The i-th sample in a dataset is denoted by a superscript, e.g., $\boldsymbol{x}^{(i)}$.
- Stochastic processes are denoted by italic capital letters with a time argument, e.g., $X(t)$, where t indicates time dependence.

Time series

- A time series of length T is denoted by $\boldsymbol{x}_{(1:T)}$, where the subscript within parentheses indicates the time index.

- When a variable is empathized as a function of time, it is denoted by $\boldsymbol{x}(t)$ for a multidimensional variable or $x(t)$ as a single one.
- The i-th dimension of a time series is denoted by a subscript, e.g., $\boldsymbol{x}_i$.

Variable modifications

- Estimated variables are denoted with a hat, e.g., $\hat{\boldsymbol{x}}$.
- Processed or transformed variables are denoted with a tilde, e.g., $\tilde{\boldsymbol{x}}$.
- Derivatives are denoted with a dot for time derivatives, e.g., $\dot{\boldsymbol{x}}$.

Probability and statistics

- Probability Density Function (PDF): $p(\cdot)$.
- Joint PDF: $p(\cdot, \cdot)$.
- Conditional PDF: $p(\cdot \mid \cdot)$.
- Expectation of a function $f(x)$ under PDF $p(x)$: $\mathbb{E}_{p(x)}[f(x)]$.
- Covariance between random vectors $\boldsymbol{x}$ and $\boldsymbol{y}$: $\mathrm{Cov}[\boldsymbol{x}, \boldsymbol{y}]$.
- Variance of a random variable $\boldsymbol{x}$: $\mathrm{Var}[\boldsymbol{x}]$.

Matrix operations

- Identity matrix: $\boldsymbol{I}$.
- Jacobian matrix: $\boldsymbol{J}$.
- Determinant of $\boldsymbol{J}$: $\det|\boldsymbol{J}|$.
- Trace of a matrix $\boldsymbol{X}$: $\mathrm{Tr}(\boldsymbol{X})$.
- Transpose of a matrix $\boldsymbol{X}$: $\boldsymbol{X}^\top$.

2.2 Bayesian Inference

Bayesian inference gets its name from using Bayes' rule to perform statistical inference of probability distributions. This chapter refers to *Bayesian Data Analysis* (Gelman et al., 2013), *Handbook of MCMC* (Allen, 2007), and *Variational Inference* (Blei et al., 2017).

2.2.1 Bayes' rule

Bayes' rule is derived from the chain rule of conditional probability. For target variables $\boldsymbol{\theta}$ and the corresponding observed data $\mathcal{D}$, the chain rule states that:

$$p(\boldsymbol{\theta}, \mathcal{D}) = p(\boldsymbol{\theta}|\mathcal{D})p(\mathcal{D}) = p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{I.2}$$

Rearranging the terms gives Bayes' rule or Bayes' theorem:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}. \tag{I.3}$$

In Bayes' theorem, $p(\boldsymbol{\theta})$ is the initial distribution of the variables before any data samples are considered, which is called the prior distribution. $p(\boldsymbol{\theta}|\mathcal{D})$ is the posterior distribution of the target parameter after being updated by the observed data. $p(\mathcal{D}|\boldsymbol{\theta})$ is the likelihood of expected data given the setting of parameters $\boldsymbol{\theta}$. $p(\mathcal{D})$ is called the marginal likelihood or evidence of expected data across all possible parameter settings:

$$p(\mathcal{D}) = \Sigma_{\boldsymbol{\theta}'} p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}', \tag{I.4a}$$

$$p(\mathcal{D}) = \int p(D|\boldsymbol{\theta}')p(\boldsymbol{\theta}')d\boldsymbol{\theta}'. \tag{I.4b}$$

In practice, it is often treated as a normalizing constant and ignored. The unnormalized posterior is then given by:

$$p_u(\boldsymbol{\theta}|\mathcal{D}) \propto p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}). \tag{I.5}$$

Normally, the posterior distribution does not have a closed-form expression. Instead, we approximate it using numerical methods. Computational algorithms that rely on repeated random sampling to obtain numerical results refers to "Monte Carlo" methods. Monte Carlo methods provide ways to approximate the posterior distribution by expectations:

$$E = \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[f(\boldsymbol{\theta})] = \int f(\boldsymbol{\theta})p(\boldsymbol{\theta}|\mathcal{D})d\boldsymbol{\theta}, \tag{I.6}$$

where $f(\boldsymbol{\theta})$ is any function of interest, for example, $f(\boldsymbol{\theta}) = \boldsymbol{\theta}$. As the simplest way, we can approximate it using standard numerical techniques such as a Riemann sum over a discrete

grid of points:

$$E \approx \Sigma_{\boldsymbol{\theta}'} f(\boldsymbol{\theta}')\Delta\boldsymbol{\theta}'. \tag{I.7}$$

However, this approach is only feasible for low-dimensional $\boldsymbol{\theta}$ due to the curse of dimensionality. Besides, for complex distributions, the discretization is highly inefficient. Therefore, for high-dimensional parameters and complex distributions, we often resort to sampling-based or optimization-based methods to approximate the posterior distribution, which will be discussed in the following sections.

2.2.2 Markov Chain Monte Carlo

The "Markov Chain" aspect indicates that these algorithms generate a sequence of samples where each sample depends only on the previous one. In contrast to the basic Monte Carlo methods, Markov Chain Monte Carlo (MCMC) methods produce auto-correlated samples, so that each new sample adds less new information than an independent draw would. Different sequences updating algorithms lead to different MCMC methods. The goal of MCMC is to construct a Markov chain that has the desired posterior distribution as its equilibrium distribution. After running the chain for a sufficient number of iterations, the samples generated from the chain can be treated as samples from the target posterior distribution.

Metropolis-Hastings (MH) algorithm

The MH algorithm, which is named after Metropolis (1953) and Hastings (1970), is a widely used MCMC method. The steps of the MH algorithm are as follows:

1. Initialize the chain with a starting point $\boldsymbol{\theta}_{(0)}$, choose a proposal distribution $q(\boldsymbol{\theta}'|\boldsymbol{\theta})$.
2. For each iteration i:
   (a) Propose a new state $\boldsymbol{\theta}'_{(i)}$ from the proposal distribution $q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)})$.
   (b) Calculate the transition probability, also known as the acceptance ratio:

$$T(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) = \min\{1, \frac{p(\boldsymbol{\theta}'_{(i)}|\mathcal{D})q(\boldsymbol{\theta}_{(i-1)}|\boldsymbol{\theta}'_{(i)})}{p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)})}\}. \tag{I.8}$$

   (c) Generate a uniform random number $u \sim U(0,1)$. Update $\boldsymbol{\theta}_{(i)}$ by "accept or reject":

$$\boldsymbol{\theta}_{(i)} = \begin{cases} \boldsymbol{\theta}'_{(i)}, & \text{if } u < T(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \\ \boldsymbol{\theta}_{(i-1)}, & \text{otherwise} \end{cases}. \tag{I.9}$$

3. Repeat step 2 for numerous iterations to obtain a sequence of samples $\{\boldsymbol{\theta}_{(1)}, \ldots, \boldsymbol{\theta}_{(N)}\}$.

From the sequence of samples, we can construct a Monte Carlo estimator:

$$p(\boldsymbol{\theta}|\mathcal{D}) \approx \frac{1}{N}\Sigma_{i=1}^{N}\delta_{\boldsymbol{\theta}_{(n)}}(\boldsymbol{\theta}). \tag{I.10}$$

Although this procedure allows flexible approximation of complex posterior distribution, it suffers from slow convergence rate due to its random exploration nature. Thus, gradient-based MCMC methods are proposed to incorporate the structure of the target density, leading to faster sampling.

Metropolis-adjusted Langevin algorithm (MALA)

The MALA (Besag, 2004) combines the MH algorithm with Langevin diffusion, whose stationary distribution is the target density. In general, for a stochastic process $\Theta(t)$, the Langevin diffusion is described by the following stochastic differential equation (SDE):

$$d\Theta(t) = \nabla \log p(\Theta(t))dt + \sqrt{2}dW(t), \tag{I.11}$$

where $W(t)$ denotes a standard Wiener process (Gabbiani and Cox, 2010). The Wiener process is a continuous-time stochastic process with independent, normally distributed increments, which can be interpreted as Brownian motion. It has the following properties:

(1) $W(0) = \mathbf{0}$;

(2) for $t > s \geq 0$, $W(t) - W(s) \sim \sqrt{t-s}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I})$;

(3) for $t_1 > s_1 \geq 0$, and $t_2 > s_2 \geq 0$, $W(t_1) - W(s_1)$ and $W(t_2) - W(s_2)$ are independent.

The MALA algorithm uses the first order Euler-Maruyama discretization of the SDE, resulting in the following proposal distribution:

$$\Theta(t) = \Theta(i-1) + \tau\nabla \log p(\Theta(i-1)) + \sqrt{2\tau}\,\mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{I.12}$$

where τ is a step size parameter. Because of the discretization, convergence to the target density is no longer guaranteed. Therefore, the MH algorithm is applied to correct the bias introduced by the discretization. For each iteration i, the MALA algorithm updates the state as follows:

(a) Compute the gradient of the log-posterior at the current state: $\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})$.

(b) Propose a new state $\boldsymbol{\theta}'_{(i)}$ using the Langevin dynamics (I.12).

(c) Calculate the transition probability according to Equation I.8. Specifically, the proposal distribution is derived from the Langevin dynamics:

$$q(\boldsymbol{\theta}'_{(i)}|\boldsymbol{\theta}_{(i-1)}) \propto \exp(-\frac{1}{4\tau}\left\|\boldsymbol{\theta}'_{(i)} - \boldsymbol{\theta}_{(i-1)} - \tau\nabla \log p(\boldsymbol{\theta}_{(i-1)}|\mathcal{D})\right\|_2^2). \tag{I.13}$$

(d) (same as MCMC) Generate a uniform random number $u \sim U(0,1)$. Update $\boldsymbol{\theta}_{(i)}$ by "accept or reject" according to Equation I.9.

Compared to the MH, MALA converges faster and scales better in higher dimensions, at the cost of requiring gradients and tuning. As a more advanced MCMC method, Hamiltonian Monte Carlo (HMC) (Duane et al., 1987) further improves sampling efficiency by simulating the whole sampling trajectory as Hamiltonian dynamics.

2.2.3 Variational Inference

Different from MCMC, Variational Inference (VI) (Jordan et al., 1998; Wainwright and Jordan, 2008) is an optimization-based method that approximates the target posterior distribution $p(\boldsymbol{\theta}|\mathcal{D})$ with a simpler, parameterized distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$, where $\boldsymbol{\phi}$ are the variational factors. The goal of VI is to find the optimal variational factors $\boldsymbol{\phi}^*$ that minimize the distance between the approximate distribution and the true posterior. The distance is normally measured by the Kullback-Leibler (KL)-divergence (Kullback and Leibler, 1951), which is defined as:

$$D_{\mathrm{KL}}(q(z)||p(z)) = \mathbb{E}_{q(z)}[\log \frac{q(z)}{p(z)}] = \int q(z) \log \frac{q(z)}{p(z)} dz. \tag{I.14}$$

Thus, the optimization problem of VI can be formulated as:

$$\boldsymbol{\phi}^* = \arg\min_{\boldsymbol{\phi}} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})). \tag{I.15}$$

Substituting Bayes' rule (I.3) into KL-divergence, we have:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||\frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}). \tag{I.16}$$

The marginal likelihood $p(\mathcal{D})$ is usually intractable, as it involves integrating over all possible parameter settings. Therefore, the divergence cannot be computed directly.

Expanding the above equation and extending it with the definition of KL-divergence,

we have:

$$\begin{aligned} D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})}[\log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \log p(\mathcal{D}|\boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) + \log p(\mathcal{D})] \\ &= \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}). \end{aligned} \tag{I.17}$$

Since $p(\mathcal{D})$ is independent of the variational factors $\boldsymbol{\phi}$, the last term is reduced to $\log p(\mathcal{D})$. Combining the terms $p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta}) = p(\boldsymbol{\theta}, \mathcal{D})$ gives

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) + \log p(\mathcal{D}) \tag{I.18}$$

Or combining the terms $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$ and $p(\boldsymbol{\theta})$ via KL-divergence gives:

$$D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) + \log p(\mathcal{D}) \tag{I.19}$$

Rearranging the terms in Equation I.18 gives:

$$\log p(\mathcal{D}) = D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta}|\mathcal{D})) + \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{I.20}$$

Since the KL-divergence is always non-negative, we have:

$$\log p(\mathcal{D}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}). \tag{I.21}$$

Because of this inequality, the right hind side is called the Evidence Lower Bound (ELBO), also known as the variational lower bound. Another way to formulate the ELBO using Equation I.19 is:

$$\mathrm{ELBO} = \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}|\boldsymbol{\theta}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta})||p(\boldsymbol{\theta})). \tag{I.22}$$

Then, minimizing the KL-divergence is equivalent to maximizing the ELBO (I.21), leading to the equivalent optimization problem:

$$\boldsymbol{\phi}^* = \arg\max_{\boldsymbol{\phi}} \mathrm{ELBO} = \arg\min_{\boldsymbol{\phi}} [\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\boldsymbol{\theta}, \mathcal{D})]. \tag{I.23}$$

To complete the optimization, we need to choose a suitable family of variational distributions $q_{\boldsymbol{\phi}}(\boldsymbol{\theta})$. A common choice is the mean-field family (Bishop, 2006), which assumes that the variables $\boldsymbol{\theta}$ are mutually independent and each governed by a distinct factor under q:

$$q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \Pi_i q_{\phi_i}(\theta_i), \tag{I.24}$$

where each factor ϕ_i governs a variable θ_i. Then, the ELBO (I.21) is iteratively optimized by updating each factor while holding the others fixed. The solution of the optimal distribution

$q_{\phi_i}(\theta_i)$ is proportional to the exponential expected log of the complete condition:

$$\log q^*_{\phi_i}(\theta_i) \propto \mathbb{E}_{\Pi_{j\neq i} q_{\phi_i}(\theta_i)}[\log p(\theta_i|\boldsymbol{\theta}_{j\neq i}, \mathcal{D})], \tag{I.25}$$

where the expectation is taken with respect to the distributions of all other groups $j \neq i$. This algorithm is called Coordinate Ascent Variational Inference (CAVI) (Bishop, 2006).

The assumption of the mean-field VI simplifies the optimization problem, and thus makes it scalable for large dimensional variables and also enables parallelizable computation. However, it may also limit the expressiveness of the variational distribution, especially when the posterior correlations are crucial, leading to suboptimal approximations of the true posterior. Summarizing, VI is generally faster than MCMC and scales better to large datasets, but it may provide less accurate approximations of the posterior distribution (Blei et al., 2017).

2.3 Deep Probabilistic Models

Deep probabilistic models combine the representational power of deep learning with the principled uncertainty modeling of probabilistic inference methods. This integration aims to provide both high expressiveness and scalability to large datasets. In the following section, we briefly review three prominent deep probabilistic models: the Deep Gaussian Process, the Bayesian Neural Network, and the Variational Autoencoder.

2.3.1 Deep Gaussian Process

A Gaussian process is a stochastic process, where at each time point the random variables follow normal distributions. It can be interpreted as an infinite-dimensional Gaussian distribution. For an N-dimensional random variable $\boldsymbol{x} = [x_1, \ldots, x_N]^T$, a Gaussian process $X(t)$ is fully specified by its mean function $\mu(\cdot)$ and covariance function $k(\cdot, \cdot)$:

$$\begin{bmatrix} X_1(t) \\ \cdots \\ X_N(t) \end{bmatrix} \sim \mathcal{N}(\begin{bmatrix} \mu(x_1) \\ \cdots \\ \mu(x_N) \end{bmatrix}, \begin{bmatrix} k(x_1, x_1), \ldots, k(x_1, x_N) \\ \cdots \\ k(x_N, x_1), \ldots, k(x_N, x_N) \end{bmatrix}) \tag{I.26}$$

Using a Gaussian process to infer variables from observed data based on prior distributions leads to Gaussian process regression (GPR) (Rasmussen and Williams, 2008). For a regression task with training data $\boldsymbol{x}$ and the ground truth $\boldsymbol{y}$, the GPR model is employed by a latent function $f(\cdot)$:

$$y_i = f(x_i) + \epsilon_i, \quad i = 1, \ldots, N, \tag{I.27}$$

where $\epsilon_i \sim \mathcal{N}(0, \sigma_\epsilon^2)$ are independent and identically distributed (i.i.d.) noises for all dimensions. We denote the collection of latent functions as $\boldsymbol{f} = [f(x_1), \ldots, f(x_N)]^T$. So, the likelihood of $\boldsymbol{y}$ is given by:

$$p(\boldsymbol{y}|\boldsymbol{f}) = \prod_{i=1}^{N} p(y_i|f(x_i)) = \prod_{i=1}^{N} \mathcal{N}(y_i|f(x_i)), \sigma_\epsilon^2). \tag{I.28}$$

The prior distribution over functions $f(\cdot)$ is assumed to a zero-mean Gaussian process. We denote it as:

$$f(\cdot) \sim \mathcal{GP}(0, k(\cdot, \cdot)). \tag{I.29}$$

The kernel function $k(\cdot, \cdot)$ is usually chosen to be the squared exponential (SE) kernel function, or as known as the Radial Basis Function (RBF), defined as:

$$k(x_i, x_j) = \sigma_{SE}^2 \exp(-\frac{1}{2\ell^2} \|x_i - x_j\|^2), \tag{I.30}$$

where σ_{SE} and ℓ are vertical and horizontal scale parameters, which control the vertical span and smoothness of the function, respectively. Then, the posterior distribution of $\boldsymbol{y}$ can be computed analytically using the marginal likelihood over $\boldsymbol{y}$:

$$p(\boldsymbol{y}|\boldsymbol{x}) = \int \prod_{i=1}^{N} p(y_i|f(x_i))p(f(x_i)|x_i)d\boldsymbol{f} = \mathcal{N}(\boldsymbol{y}|\boldsymbol{0}, \boldsymbol{K} + \sigma_\epsilon^2 \mathbb{I}), \quad \boldsymbol{K} = k(\boldsymbol{x}, \boldsymbol{x}). \tag{I.31}$$

The hyperparameters $\boldsymbol{\phi} = \{\sigma_{SE}, \ell, \sigma_\epsilon\}$ can be optimized by maximizing the log marginal likelihood $\log p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\phi})$.

The GPR is widely used for regression tasks owing to its non-parametric nature and its capacity to quantify uncertainty. However, its computational complexity scales cubically with the number of training points, i.e., $\mathcal{O}(N^3)$ for an N-dimensional input. Additionally, calculating the Gaussian processes requires the entire storage of training datasets, incurring a memory cost of $\mathcal{O}(M^2)$, which grows quadratically with the dataset size M. These limitations hinder its scalability to high-dimensional inputs and large datasets. In addition, the expressiveness of GPR is constrained by the choice of kernel function and its incapacity to perform hierarchical feature extraction.

To improve expressiveness, DGP (Damianou and Lawrence, 2013) is proposed as multilayer hierarchical generalizations of Gaussian processes. The input $\boldsymbol{x}$ is passed through multiple layers of Gaussian processes to obtain the output $\boldsymbol{y}$. For an L-layer DGP,

the model extends the Gaussian process (I.27) by:

$$y_i = f_L(f_{L-1}(\dots f_1(x_i) + \epsilon_{i,1} \dots)) + \epsilon_L, \quad f_l(\cdot) \sim \mathcal{GP}(0, k_l(\cdot,\cdot)), \\ l = 1, \dots, L,\ i = 1, \dots, n. \quad \text{(I.32)}$$

The deep architecture enables Gaussian processes to learn more complex data patterns and hierarchical representations. It can be seen as a deep neural network with infinitely wide layers (Neal, 1996), where each layer applies a Gaussian process, instead of nonlinear matrix transformation. However, due to the computational cost and storage consumption the DGP is still cumbersome to apply in practice.

2.3.2 Bayesian Neural Networks

Another approach to constructing deep probabilistic models involves integrating Bayesian inference with deep neural networks. A Bayesian Neural Network (BNN) (Neal, 1996; MacKay, 1995) places prior distributions over the network's weights and biases, allowing uncertainty to be modeled directly within the architecture.

The goal of training a BNN is to infer the posterior distribution over the parameters $\boldsymbol{\theta} = [\boldsymbol{w}, \boldsymbol{b}]$ given the training data $\mathcal{D}$, i.e., $p(\boldsymbol{\theta}|\mathcal{D})$. VI is commonly employed to approximate the posterior by optimizing the ELBO (I.22). To facilitate gradient-based optimization via backpropagation—similar to standard neural networks—the reparameterization trick (Kingma and Welling, 2022) is used. This technique expresses random variables as deterministic variables of noise, enabling efficient and differentiable sampling during training. For example, for a Gaussian variational distribution $q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\theta}|\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$, we can reparameterize $\boldsymbol{\theta}$ as:

$$\boldsymbol{\theta} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \quad \text{(I.33)}$$

where $\odot$ represents the Hadamard product, i.e., the element-wise product, and the variational parameters here are $\boldsymbol{\phi} = [\boldsymbol{\mu}, \boldsymbol{\sigma}]$. To train a BNN on large datasets, Stochastic Gradient Descent (SGD) (Bottou, 1999) is typically employed to optimize the ELBO. Unlike standard gradient descent, SGD estimates the gradient at each iteration using a randomly selected subset of the data. Then, the variational objective of a BNN using the ELBO (I.22) becomes:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = \frac{1}{M} \sum_{i=1}^{M} [D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{\theta}) || p(\boldsymbol{\theta})) - \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{\theta})} \log p(\mathcal{D}_i|\boldsymbol{\theta})], \quad \text{(I.34)}$$

where each mini-batch $\mathcal{D}_i \subset \mathcal{D}$ is drawn from the equally sized subsets of the training data $\mathcal{D} = \{\mathcal{D}_1, \dots, \mathcal{D}_M\}$. During each optimization step, the auxiliary noise $\boldsymbol{\epsilon}$ is sampled

multiple times from $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ to perform Monte Carlo estimation of the expectation terms in the loss function $\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$. The gradient $\nabla_{\boldsymbol{\phi}}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi})$ is then computed via backpropagation and averaged across all samples. Finally, the variational parameter $\boldsymbol{\phi}$ is updated by

$$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha\nabla_{\boldsymbol{\phi}}\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}), \tag{I.35}$$

where α is the learning rate. This procedure is called Bayes by Backprop (BbB) (Blundell et al., 2015).

In comparison to DGP, the BNN demonstrates better scalability to large datasets and improved representational capacity. Moreover, the BbB method facilitates the compatibility of BNN with deep learning techniques and thus enables flexible implementation in different neural network architectures, such as Convolutional Neural Network (CNN) (Shridhar et al., 2019), RNN (Fortunato et al., 2019), and Transformer (Müller et al., 2024). Nevertheless, several works raise concerns with BNNs including wrong uncertainty estimation in between separated regions of observations (Foong et al., 2019, 2020), poor generalization ability under domain shift (Izmailov et al., 2021), and sensitive modeling ability to the choice of prior distributions (Silvestro and Andermann, 2020). These limitations motivate the applications of more powerful generative models, such as variational autoencoders and NFs, which are discussed in the following sections.

2.3.3 Variational Autoencoder

In classical VI methods, the parameters of interest $\boldsymbol{\theta}$ are random variables, and the approximate posterior distribution over the parameters is parameterized by variational parameters $\boldsymbol{\phi}$. In contrast, a Variational Autoencoder (VAE) is constructed with deterministic neural networks and introduces variability by latent variables. For a dataset $\mathcal{D} = \{\boldsymbol{x}^{(i)}\}_{i=1}^{N}$, the samples are assumed to be i.i.d. and drawn from an unchanging underlying distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$. A VAE extracts information from input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$ by a parameterized inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, which is also called an encoder or recognition model. Then, the VAE generates data $\boldsymbol{x}$ from the latent variables $\boldsymbol{z}$ by another parameterized model $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, which is also called a decoder. The decoder is often mentioned as a "generative" model. However, according to the formal definition (Kingma and Welling, 2019), a generative model learns a joint distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{z})p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$, including sampling $\boldsymbol{z} \sim p_{\boldsymbol{\theta}}(\boldsymbol{z})$ and sampling $\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$.

The latent variables connect the inference model and the generative model. From the encoder side, the inference model $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ approximates the true posterior distribution $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ of the generative model, which is intractable. From the decoder side, the distribution of latent variables is a prior distribution, which is usually chosen to be a standard normal distribution, i.e., $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\mathbf{0}, \boldsymbol{I})$. The whole architecture is illustrated in Figure I.1.

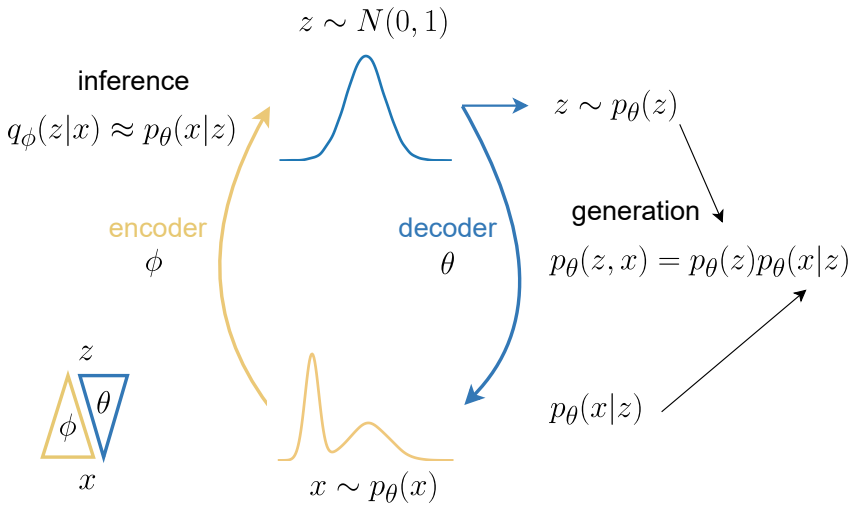


Figure I.1: The architecture of a Variational Autoencoder. The encoder (with neural network parameters $\boldsymbol{\phi}$) maps the input data $\boldsymbol{x}$ to latent variables $\boldsymbol{z}$, and the decoder (with neural network parameters $\boldsymbol{\theta}$) reconstructs the input data from the latent variables.

Same reparameterization trick as in the BNN is used to enable gradient-based optimization. The latent variables $\boldsymbol{z}$ becomes:

$$\boldsymbol{z} = \boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}), \tag{I.36}$$

where $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\boldsymbol{x})$ and $\boldsymbol{\sigma}_{\boldsymbol{\phi}}(\boldsymbol{x})$ are also neural networks. Different from the mini-batch optimization in the BNN, we use one single encoder neural network to perform posterior inference over all samples in the dataset. This strategy of sharing variational parameters across datapoints is called amortized variational inference (Gershman and Goodman, 2014; Margossian and Blei, 2024), which improves the optimization efficiency.

The objective of the inference model is to minimize the KL-divergence between the inferred and true posterior $D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}))$. Recall the ELBO in VI (I.21, I.22), the ELBO of a VAE is given by:

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) - D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p_{\boldsymbol{\theta}}(\boldsymbol{z})). \tag{I.37}$$

Maximizing the ELBO optimizes concurrently both the inference model and the generative model. The marginal likelihood $p_{\boldsymbol{\theta}}(\boldsymbol{\theta})$ is maximized, meaning the generated data maximally resembles the input data. Meanwhile, the distance between the approximated and the true posterior is minimized. Changing the prior distribution in the ELBO, $p_{\boldsymbol{\theta}}(\boldsymbol{z}) := p(\boldsymbol{z})$, the

overall loss function of a VAE is given by:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}) = -\mathbb{E}_{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) + D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z})). \tag{I.38}$$

The first term is a reconstruction loss, which encourages the decoder to reconstruct the input data from the latent variables. The second term is a regularization term, which forces the approximate posterior to be close to the prior distribution.

VAEs have proven to be versatile and powerful in diverse applications, such as representation learning, anomaly detection, and generative modeling (Mienye and Swart, 2025). To enable VAEs to perform probabilistic inference of variables $\boldsymbol{x}$ given observations $\boldsymbol{y}$, the observations should be inserted into both the encoder and decoder as conditions. This method is named as conditional VAE (Sohn et al., 2015). The inference model becomes $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{y})$, the prior distribution becomes $p(\boldsymbol{z}|\boldsymbol{y})$, and the generative model becomes $p_{\boldsymbol{\theta}}(\boldsymbol{z}, \boldsymbol{x}|\boldsymbol{y})$. The conditions are often constrained to be low-dimensional variables, such as class labels (Sohn et al., 2015) or physical parameters (Martínez-Palomera et al., 2020). For high-dimensional and complex conditions, a powerful decoder is required to capture the relationship between the variables and observations. In this case, the latent variables may be ignored and thus the variability losses. This issue is known as posterior collapse (Bowman et al., 2016; McCarthy et al., 2020), which limits the application of VAEs in conditional inference tasks.

2.4 Normalizing Flows

Rezende and Mohamed (2016) proposed Normalizing Flow (NF) as a method for approximating posterior distributions by transforming a simple base distribution through a sequence of invertible mappings into complex target distributions.

2.4.1 Basics

We simplify a NF as an invertible and differentiable function $f : \mathbb{R}^D \to \mathbb{R}^D$ (as known as a diffeomorphism) that maps a random variable $\boldsymbol{z}$ in latent space to another random variable $\boldsymbol{x}$ in data space:

$$\boldsymbol{x} = f(\boldsymbol{z}), \tag{I.39a}$$

$$\boldsymbol{z} = f^{-1}(\boldsymbol{x}). \tag{I.39b}$$

The variable $\boldsymbol{x}$ follows an underlying distribution of data $p_X(\boldsymbol{x})$. The variable $\boldsymbol{z}$ is drawn from a simple base distribution $p_Z(\boldsymbol{z})$, such as a standard normal distribution. Understanding the transformation between the two density functions is essential for comprehending NFs.

Change of Variables

According to the change of variables formula (Schwartz, 1954), when both the latent and data space are smooth and the function f is differentiable, we have:

$$\int \Psi(\boldsymbol{x})d\boldsymbol{x} = \int \Psi(\boldsymbol{x} = f(\boldsymbol{z})) \det |\boldsymbol{J}_f(\boldsymbol{z})| \, d\boldsymbol{z}, \tag{I.40}$$

where Ψ is a continuous function, $\boldsymbol{J}_f(\boldsymbol{z})$ is the Jacobian of the mapping f and $\det|\cdot|$ stands for determinant:

$$\det |\boldsymbol{J}_f(\boldsymbol{z})| = \det \left| \frac{\partial f(\boldsymbol{z})}{\partial \boldsymbol{z}} \right|. \tag{I.41}$$

This equation (I.40) implies probability conservation under transformation. In probability theory, the Radon–Nikodym derivative (Durrett, 1996) is a way to express how one probability measure changes with respect to another, which can be seen as a generalization of the relationship between Cumulative Distribution Function (CDF) and PDF. For a measurement set A for the variable $\boldsymbol{x}$, the distribution is:

$$P(X \in A) = \int_A p_X(\boldsymbol{x})d\boldsymbol{x}. \tag{I.42}$$

Applying the Equation I.39b, we have:

$$P(X \in A) = P(Z \in f^{-1}(A)), \tag{I.43a}$$

$$\int_A p_X(\boldsymbol{x})d\boldsymbol{x} = \int_{f^{-1}(A)} p_Z(\boldsymbol{z})d\boldsymbol{z}. \tag{I.43b}$$

Using change of variables formula (I.40), the last integral becomes:

$$\int_{f^{-1}(A)} p_Z(\boldsymbol{z})d\boldsymbol{z} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| d\boldsymbol{x}, \tag{I.44}$$

where the Jacobian determinant of the inverse mapping f^{-1} is:

$$\det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| = \det \left| \frac{\partial f^{-1}(\boldsymbol{x})}{\partial \boldsymbol{x}} \right| = \det \left| \frac{\partial \boldsymbol{z}}{\partial f(\boldsymbol{z})} \right| = \frac{1}{\det |\boldsymbol{J}_f(\boldsymbol{z})|}. \tag{I.45}$$

Fusing the equations I.43b and I.44 together, we get the integral equality on the measurement set A:

$$\int_A p_X(\boldsymbol{x})d\boldsymbol{x} = \int_A p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| d\boldsymbol{x}. \tag{I.46}$$

Since the integrands are continuous almost everywhere, the derivatives also hold the equality:

$$p_X(\boldsymbol{x}) = p_Z(f^{-1}(\boldsymbol{x})) \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| . \tag{I.47}$$

Substituting $\boldsymbol{x}$ and $\boldsymbol{z}$ with equations I.39a and I.39b respectively, it can also be formulated using the forward mapping f:

$$p_Z(\boldsymbol{z}) = p_X(f(\boldsymbol{z})) + \det \left| \boldsymbol{J}_f(\boldsymbol{z}) \right| . \tag{I.48}$$

These two equations are declared as the change of variables applied for NFs. They are often formulated in logarithm function:

$$\log p_X(\boldsymbol{x}) = \log p_Z(f^{-1}(\boldsymbol{x})) + \log \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}) \right| , \tag{I.49}$$

$$\log p_Z(\boldsymbol{z}) = \log p_X(f(\boldsymbol{z})) + \log \det \left| \boldsymbol{J}_f(\boldsymbol{z}) \right| . \tag{I.50}$$

From the above derivation of the formulas, we can infer three important properties of NFs:

(1) The transformation function f is differentiable.

(2) The transformation function f is invertible. Or from the definition in set theory, f is bijective, which can be interpreted as globally invertibility.

(3) The Jacobian determinant should be efficient to compute.

On top of them, another important property the transformation f should be fulfilled is composability (Papamakarios et al., 2021). Given a series of transformations $f_L \circ \cdots \circ f_1$, the following equations hold:

$$\boldsymbol{x} = f_L(\cdots f_1(\boldsymbol{z})), \tag{I.51a}$$

$$(f_L \circ \cdots \circ f_1)^{-1} = (f_1)^{-1} \circ \cdots \circ (f_L)^{-1}, \tag{I.51b}$$

$$\boldsymbol{J}_{f_L \circ \cdots \circ f_1}(\boldsymbol{z}) = \boldsymbol{J}_{f_L}(f_{L-1}(\boldsymbol{z})) \cdots \boldsymbol{J}_{f_1}(\boldsymbol{z}), \tag{I.51c}$$

where the total Jacobian determinant is the product of each intermediate ones.

Using NFs for modeling and inference

The invertibility also provides us easy way to fit NF models, i.e., minimizing the divergence between the true data distribution and the estimated one. For a NF with parameters $\boldsymbol{\theta}$, the KL-divergence is given by:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x}) || p_X(\boldsymbol{x}; \boldsymbol{\theta})) = \mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x}) - \mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x}; \boldsymbol{\theta}). \tag{I.52}$$

The first term can be considered constant while optimizing the parameters $\boldsymbol{\theta}$. The second term can be estimated by Monte Carlo methods:

$$\mathbb{E}_{p_X(\boldsymbol{x})} \log p_X(\boldsymbol{x};\boldsymbol{\theta}) \approx \frac{1}{M} \sum_{m=1}^{M} \log p_X(\boldsymbol{x}^{(m)};\boldsymbol{\theta}), \tag{I.53}$$

using a dataset $[\boldsymbol{x}^{(m)}]$, $m = 1, \ldots, M$. This is equivalent to the maximum likelihood estimation. Expanding $p_X(\boldsymbol{x};\boldsymbol{\theta})$ with Equation I.47, the KL-divergence becomes:

$$D_{\mathrm{KL}}(p_X(\boldsymbol{x})||p_X(\boldsymbol{x};\boldsymbol{\theta})) \approx const.- \\ \frac{1}{M} \sum_{m=1}^{M} [\log p_Z(f^{-1}(\boldsymbol{x}^{(m)};\boldsymbol{\theta})) + \log \det \left| \boldsymbol{J}_{f^{-1}}(\boldsymbol{x}^{(m)};\boldsymbol{\theta}) \right|], \tag{I.54}$$

where $p_Z(\boldsymbol{z})$ is a standard normal distribution. Then, the parameters are optimized by stochastic gradient-based methods.

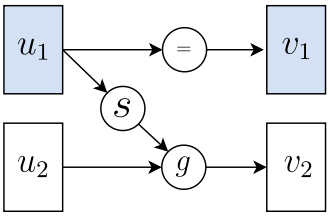
The loss function of NFs suggests that during training, the learned latent representation is compared with the true latent distribution, while the term of Jacobian determinant ensures this comparison to be transformed back to the data space. Recalling the VAE, the first term can be seen as inference, while the second term can be implicitly regarded as "preparation for generation". After the NF model is well-trained, new data can be generated by sampling the latent variables and transforming them via the forward mapping. It is important to emphasize that, NFs are conventionally trained in one direction at a time. Simultaneously bidirectional training would necessitate computing gradients through the composition of a function and its inverse, leading to problematic gradient flows of the form $\partial L/\partial \boldsymbol{\theta}_f(\boldsymbol{\theta}_{\boldsymbol{f^{-1}}})$, which creates numerical instabilities and computational inefficiencies.

A variety of architectures have been developed to realize NFs. A GitHub website (Riebesell et al., 2023) summarizes the key publications and implementations in this domain. As outlined in the comprehensive review by Kobyzev et al. (2021), successful architectures include coupling flows, autoregressive flows, residual flows, and infinitesimal (continuous) flows. Throughout this work, we focus primarily on coupling flows, which offer structural invertibility and have been extensively utilized in NFs. Infinitesimal flows characterize distributional transformations through continuous dynamics, establishing deep theoretical connections with other probabilistic frameworks such as diffusion models. Thus, we also introduce continuous NFs following the description of coupling flows.

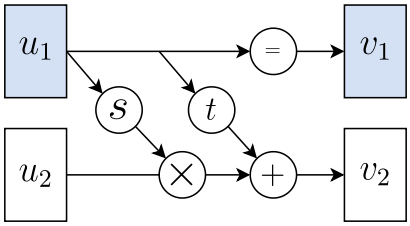
2.4.2 Coupling Flows

The basic idea of the coupling flow is partitioning the input variables into two subsets, one subset of which is processed by neural networks and also concurrently directly passed

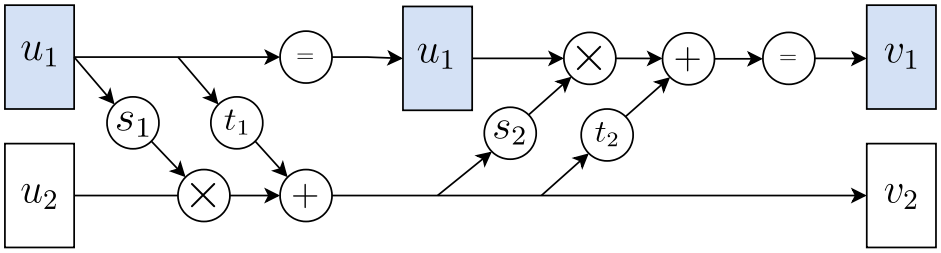
through to the output variables, while the other subset is combined with the processed subset through affine functions.



(a) The architecture of a Non-linear Independent Component Estimation (NICE) layer. s is a neural network, and g is an affine function.



(b) The architecture of a Real-Valued Non-Volume Preserving (RealNVP) layer. s and t are neural networks. The symbol × stands for element-wise product by the exponential function over input.



(c) The architecture of an Invertible Neural Network (INN) layer. s_1, s_2, t_1, and t_2 are neural networks. The symbol × stands for element-wise product by the exponential function over input.

Figure I.2: The architectures of coupling flows. The input of the layer is $\boldsymbol{u}$ and the output is $\boldsymbol{v}$.

The first proposed coupling flow is NICE (Dinh et al., 2015). The input variables $\boldsymbol{u}$ are split into two parts $[\boldsymbol{u}_1, \boldsymbol{u}_2]$, and the corresponding output variables are $\boldsymbol{v} = [\boldsymbol{v}_1, \boldsymbol{v}_2]$. The architecture in the forward direction is formulated by the following functions:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = g(\boldsymbol{u}_2; s(\boldsymbol{u}_1)), \tag{I.55}$$

where s is a neural network and g is an affine function $g(a; b) = a + b$ or $a \odot b$. The forward architecture is illustrated in Figure I.2a. Then, the inverse functions are easily calculated:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = g^{-1}(\boldsymbol{v}_2; s(\boldsymbol{u}_1)). \tag{I.56}$$

From the above functions, the Jacobian matrix is:

$$\frac{\partial \boldsymbol{v}}{\partial \boldsymbol{u}} = \begin{bmatrix} \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_1/\partial \boldsymbol{u}_2 \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_2 \end{bmatrix} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{0} \\ \partial \boldsymbol{v}_2/\partial \boldsymbol{u}_1 & \partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2 \end{bmatrix} \tag{I.57}$$

Because of the triangle matrix, the determinant is $\partial g(\boldsymbol{u}_2; s(\boldsymbol{u}_1))/\partial \boldsymbol{u}_2$. By composing multiple coupling layers, we can construct increasingly complex hierarchical transformations. Since each coupling layer preserves a subset of its input unchanged, it is essential to alternate the roles of the partitioned subsets across successive layers. This ensures that all input dimensions are eventually processed. Analyzing the Jacobian reveals that a minimum of three coupling layers is required for full inter-dimensional interaction. In practice, we typically employ four layers to achieve sufficient expressiveness.

An improved architecture is RealNVP, where one part of the variables is processed by two neural networks. The forward transformation is:

$$\boldsymbol{v}_1 = \boldsymbol{u}_1, \quad \boldsymbol{v}_2 = \boldsymbol{u}_2 \odot \exp(s(\boldsymbol{u}_1)) + t(\boldsymbol{u}_1). \tag{I.58}$$

And the inverse transformation is:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \quad \boldsymbol{u}_2 = (\boldsymbol{v}_2 - t(\boldsymbol{u}_1)) \odot \exp(-s(\boldsymbol{u}_1)). \tag{I.59}$$

The Jacobian determinant $\det |\partial \boldsymbol{v}/\boldsymbol{u}|$ is then $exp(\Sigma_i s(\boldsymbol{u}_{1,i}))$. The architecture is shown in Figure I.2b. To interact all variables in one block, INN (Ardizzone et al., 2019a) extends RealNVP by another same coupling block, as shown in Figure I.2c.

A further enhancement of RealNVP is Glow (Kingma and Dhariwal, 2018), which inserts an invertible normalization layer and an invertible permutation layer before affine transformation. Hierarchical Invertible Neural Transport (HINT) (Kruse et al., 2021) partitions the inputs of coupling blocks recursively to construct a dense, triangular Jacobian. Another method to increase interaction among variables is Masked Autoregressive Flow (MAF) (Papamakarios et al., 2018), which can be considered as a generalized RealNVP that leverages autoregressive transformations between variables.

Such coupling flows and autoregressive flows build step-wise invertible transformations between distributions, enabling tractable Jacobian calculation and density estimation. A notable drawback lies in the strict architectural constraint on coupling blocks. To release this limitation, residual flows (Behrmann et al., 2019; Chen et al., 2020) introduce invertible mapping through specified residual blocks. A residual block (He et al., 2015) takes the form $f(x) = x + F(x)$, where F is a neural network. When F is constrained to be Lipschitz continuous and the Lipschitz constant is smaller than 1 (explained in 2.4.3), the function $f(x)$ is invertible. The residual block can be seen as an Euler discretization

of an ordinary differential equation (ODE) (Lu et al., 2020; Ruthotto and Haber, 2018; Haber and Ruthotto, 2017), which is the foundation of continuous NFs.

2.4.3 Continuous Flows

Neural ODEs

For a dynamical system with states $\boldsymbol{h}$, an ODE is:

$$\frac{d\boldsymbol{h}(t)}{dt} = f(\boldsymbol{h}(t), t; \boldsymbol{\theta}), \tag{I.60}$$

where $\boldsymbol{\theta}$ denotes system parameters. When the ODE is modeled by neural networks, it becomes a Neural ODE (Chen et al., 2019). Given the initial state $\boldsymbol{h}(0)$, the final state $\boldsymbol{h}(T)$ is the solution of the initial value problem at the time step T using a numerical solver. The loss function of training a Neural ODE is:

$$L(\boldsymbol{h}(T)) = L(\int_0^T f(\boldsymbol{h}(t), t; \boldsymbol{\theta})dt), \tag{I.61}$$

where f is a neural network parameterized by $\boldsymbol{\theta}$. A straightforward application of Neural ODEs is learning continuous time-series models. Moreover, when the time step is interpreted as architectural depth, the distributional transformation in NFs can also be constructed with a Neural ODE. The Neural ODE replaces an infinite number of transformations by change of variables with a single neural network, which represents the derivative of transformation at each depth. This enables more efficient and expressive probabilistic modeling and inference.

A central technical challenge in training continuous-depth networks lies in executing reverse-mode differentiation (commonly known as backpropagation) through the ODE solver. This process requires computing gradients with respect to network parameters at various time points backwards from $t = T$ to $t = 0$, resulting in large memory cost to store the intermediate states. The solution is using the adjoint method (Bittner, 1963; Gavrilović et al., 1963). The principle of the adjoint method is to solve a backward ODE alongside the forward ODE whenever intermediate states are required. Compared to storing all intermediate states of the forward ODE, the adjoint method reduces memory cost at the expense of increased computational cost. This trade-off makes it more suitable for long sequences and high-dimensional variables, but less efficient for computationally complex network architectures.

Lipschitz continuity

We define a continuous NF as a Neural ODE:

$$\frac{d\boldsymbol{z}(t)}{dt} = f(\boldsymbol{z}, t; \boldsymbol{\theta}), \tag{I.62}$$

where $\boldsymbol{z}(t)$ follows the density $p(\boldsymbol{z}(t))$ at an instantaneous step $t \in [t_0, t_1]$. The NF f is parameterized by $\boldsymbol{\theta}$. To apply the Neural ODE as continuous NFs, the function f should be uniformly Lipschitz continuous in the states $\boldsymbol{z}(t)$ and continuous in depth t.

The definition of a Lipschitz continuous function is referred to the work by Khromov and Singh (2024).

Definition 1 ***(Lipschitz continuous function)*** *A function $f : \mathbb{R}^d \to \mathbb{R}^K$, defined on some domain $dom(f) \subseteq \mathbb{R}^d$, is called C-Lipschitz continuous w.r.t. some α-norm, if $\forall \boldsymbol{x}, \boldsymbol{y} \in dom(f) : \|f(\boldsymbol{x}) - f(\boldsymbol{y})\|_\alpha \leq C \|\boldsymbol{x} - \boldsymbol{y}\|_\alpha$. C is called Lipschitz constant of f and $0 < C < \inf$.*

The Lipschitz condition guarantees that there exists a unique solution curve $\boldsymbol{t}$ for every initial condition $\boldsymbol{z}(0)$. This is known as the Picard–Lindelöf theorem. Assuming the dynamics function f is smooth, then it is a diffeomorphism, which means we also get the same $\boldsymbol{z}(0)$ by running the ODE backwards. Recalling the residual flows, where the Lipschitz constant must be smaller than 1, invertibility is ensured naturally by a smooth and Lipschitz continuous ODE. In practice, neural networks with smooth continuous activation functions satisfy this requirement.

Continuous NFs

Besides for the ODE I.62, the log-density of states changes according to another ODE, the instantaneous change-of-variables formula (Chen et al., 2019):

$$\frac{\partial \log p(\boldsymbol{z}(t))}{\partial t} = -\mathrm{Tr}\left(\frac{\partial f(\boldsymbol{z}(t), t; \boldsymbol{\theta})}{\partial \boldsymbol{z}(t)}\right). \tag{I.63}$$

The trace can be approximated through Skilling-Hutchinson trace estimator (Skilling, 1989; Hutchinson, 1990; Adams et al., 2018):

$$\mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) = \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[\boldsymbol{\epsilon}^T \frac{\partial f}{\partial \boldsymbol{z}(t)} \boldsymbol{\epsilon}\right], \tag{I.64}$$

where $\boldsymbol{\epsilon}$ is a noise vector sampled from a standard normal distribution and kept unchanged to maintain determinism of the ODE. The part of vector-Jacobian product $\boldsymbol{\epsilon}^T \partial f / \partial \boldsymbol{z}(t)$ is computed with automatic differentiation.

For a continuous NF between the data distribution $p(\boldsymbol{x}) = p(\boldsymbol{z}(t_1))$ and the latent distribution $p_{Z_0}(\boldsymbol{z}_0) = p(\boldsymbol{z}(t_0)) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, we define the augmented states as $[\boldsymbol{z}(t), \log p(\boldsymbol{x}) - \log p(\boldsymbol{z}(t))]$. Then, building the continuous NF is equivalent to solving the initial value problem (Grathwohl et al., 2018):

$$\underbrace{\begin{bmatrix} \boldsymbol{z}_0 \\ \log p(\boldsymbol{x}) - \log p_{Z_0}(\boldsymbol{z}_0) \end{bmatrix}}_{\text{solutions}} = \int_{t_1}^{t_0} \underbrace{\begin{bmatrix} f(\boldsymbol{z}(t), t; \theta) \\ -\mathrm{Tr}\left(\frac{\partial f}{\partial \boldsymbol{z}(t)}\right) \end{bmatrix}}_{\text{dynamics}} dt, \underbrace{\begin{bmatrix} \boldsymbol{z}(t_1) \\ \log p(\boldsymbol{x}) - \log p(\boldsymbol{z}(t_1)) \end{bmatrix}}_{\text{initial values}} = \begin{bmatrix} \boldsymbol{x} \\ 0 \end{bmatrix}. \quad \text{(I.65)}$$

It is worth noting that the integral is backwards calculated from t_1 to t_0, meaning from the data side to the latent side. The time span can also be optimized to further improve the computational efficiency (Du et al., 2022).

Continuous NFs release the architectural constraints of discrete flows, allowing more flexible and expressive density estimation through the integration of Neural ODEs. This improved expressiveness, however, comes at a cost: computing the Jacobian trace and the associated ODE integrals is significantly more expensive than in stacked coupling flows.

2.4.4 Conditional Normalizing Flows

Such capabilities of estimating complex densities are desirable in Bayesian inference of the posterior distribution of variables given observations $p(\boldsymbol{x}|\boldsymbol{y})$. Similar to VAE, amortized Bayesian inference (Radev, 2021) is applied in NF to achieve a general posterior estimation across all datapoints. Obtaining the posterior distribution is represented by a conditional density transformation via a conditional NF. The change of variables for a conditional NF extends the standard one (I.47) by the condition $\boldsymbol{y}$:

$$p_{X|Y}(\boldsymbol{x}|\boldsymbol{y}) = p_{Z|Y}(f^{-1}(\boldsymbol{x}, \boldsymbol{y})|\boldsymbol{y}) \det \left| \frac{\partial f^{-1}(\boldsymbol{x}, \boldsymbol{y})}{\partial \boldsymbol{x}} \right|, \quad \text{(I.66)}$$

where the mapping $f : Z \to X \times Y$ is bijective between X and Z, and $\boldsymbol{z} = f^{-1}(\boldsymbol{x}, \boldsymbol{y})$.

Designing such a conditional NF requires addressing two key considerations in advance: (1) how to manage the conditioned latent variables, and (2) how to incorporate conditions into the flow architecture while preserving invertibility. Winkler et al. (2023) proposed to build the conditional prior $p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y})$ as a diagonal Gaussian:

$$p_{Z|Y}(\boldsymbol{z}|\boldsymbol{y}) = \mathcal{N}\boldsymbol{z}|\mu(\boldsymbol{y}, \mathrm{diag}(\sigma^2(\boldsymbol{y}))). \quad \text{(I.67)}$$

For simpler conditioning scenarios, incorporating the conditional prior could accelerate the transformation through leveraging the information from conditions. For complex conditions, summarizing the conditions through mean and variance may result in information loss,

thereby degrading posterior estimation quality. This limitation likely explains why most existing work on conditional normalizing flows adopts a standard Gaussian as the latent base distribution, bypassing the need for an explicitly conditioned prior and relying instead on the normalizing flows to capture conditional information.

In terms of architectures of conditional NFs, a direct adaptation based on the INN is the conditional Invertible Neural Network (cINN) (Ardizzone et al., 2019b). It takes the condition as augmented input to the neural networks. In cases of higher-dimensional conditions, such as images, Anantha Padmanabha and Zabaras (2021) apply a staged cINN coupled with a staged condition network, wherein the conditions are progressively processed, and each intermediate representation is fed into its corresponding cINN block.

In scenarios involving complex conditions and relatively simple target variables, BayesFlow (Radev et al., 2020) offers a general framework by extracting informative representations through a summary network and incorporating them, alongside the inputs, as augmented inputs to the cINN. This framework is particularly well-suited for parameter identification from time series observations, where the summary network is typically chosen as a time series modeling architecture, such as RNN. Both the summary network and cINN are optimized together by minimizing the KL-divergence between the true posterior density and the estimated one (I.54). The joint optimization approach encourages the summary network to deliver maximally useful information to the cINN thus improve the inference capability.

2.5 From Inference to Control

This section begins by reviewing traditional techniques for system identification and state estimation applying Bayesian inference. Subsequently, we present classical model-free control approaches and a commonly employed model-based control method, MPC.

2.5.1 Bayesian System Identification

2.5.2 Bayesian State Estimation

2.5.3 Model-free Control

2.5.4 Model Predictive Control

3 Literature Review

This section reviews the relevant literature in three aspects: applications of deep probabilistic models especially normalizing flows for Bayesian inference; state-of-the-art deep probabilistic methods for system identification and state estimation; and data-driven control methods based on deep probabilistic models.

3.1 Deep Probabilistic Models for Bayesian Inference

3.2 Conditional Normalizing Flows

3.3 Deep Probabilistic Models for System Identification and State Estimation

3.4 Data-driven Control based on Deep Probabilistic Models

4 Objectives and Contributions

Starting from BayesFlow, we explore the potential of normalizing flows for parameter identification, system identification, and state estimation. Moreover, we aim to bridge the gap between probabilistic inference and control tasks. The primary objectives of this dissertation are as follows:

- To investigate the capabilities of normalizing flows for system parameter identification.
- To extend BayesFlow for paramter identification to perform system identification and state estimation tasks.
- To assess the performance of the proposed models on benchmark datasets and real-world scenarios.
- To analyze the interpretability and uncertainty quantification capabilities of the learned models.
- To examine the applicability of the identified systems in control tasks.

The key contributions of this dissertation can be summarized as:

- Enhancement of the original BayesFlow framework to improve robustness, enabling its deployment in real-world applications.
- Integration of BayesFlow with sensitivity analysis to facilitate the estimation of parameters with different degrees of identifiability.
- Development of Stochastic BayesFlow by combining stochastic normalizing flows with BayesFlow, aimed at robust parameter identification under uncertainty.
- Extension of BayesFlow to learn sequence-to-sequence models for system identification and state estimation.
- Proposal of FlowSDE, a new method for learning unknown stochastic dynamical systems by embedding normalizing flows within stochastic differential equations.
- Investigation of the compatibility of normalizing flow-based models within control frameworks.

5 Outline

The remainder of this thesis is organized as follows:

- Chapter II outlines the methodologies developed and employed throughout this work. These methods are grouped into three categories: parameter identification, dynamics identification, and state estimation. Finally, data-driven control strategies based on dynamics identification approaches are introduced.
- Chapter III presents the experimental validation of the proposed methods, structured according to the aforementioned categories.
- Chapter IV summarizes the key findings of the dissertation and proposes directions for future research.

Chapter II

Methodologies

1 Parameter Identification

BayesFlow

- Stochastic BayesFlow
- BayesFlow for differently identifiable parameters

2 Dynamics Identification

IMFlow
- FMFlow
- FlowSDE

3 State Estimation

IMFlow ability limited

limitation for bayesflow for identifying highly dynamical data

transformer-bayesian neural network better for identification of battery state-of-health

Luenberger Observer with FlowSDE

4 Data-driven Control

Model free control with IMFlow

Model predictive control with FMFlow

Model predictive control with FlowSDE

Chapter III

Experiments and Results

1 Overview

2 Parameter Identification

von der pol single track car quadcopter

3 Dynamics Identification

synthetic data physics-based real-world

4 State Estimation

5 Data-driven Control

model-free control

model predictive control

Chapter IV

Discussion and Conclusion

Chapter V

Abstract

Chapter VI

Zusammenfassung

Own Contributions

This chapter gives an overview of my contributions in distinction to team efforts.

Own share in data acquisition and data analysis

Own share in Data Acquisition.

Own share in Data Analysis.

Own Publications

In this section, all publications that I was a part of and contributed to during my Ph.D. work are listed. It is subdivided into *First Authorships* and *Co-Authorships*.

First Authorships - Peer Reviewed International Journal Publications

Y. Zhang, L. Mikelsons. **Advanced Modeling and Simulation in Engineering and Science**. *Sensitivity-guided iterative parameter identification and data generation with BayesFlow and PELS-VAE for model calibration.* doi:10.1186/s40323-023-00246-y.

First Authorships - Peer Reviewed International Conference Publications

Y. Zhang, L. Mikelsons. **2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)**. *Solving Stochastic Inverse Problems with Stochastic BayesFlow.* doi:10.1109/AIM46323.2023.10196190.

Y. Zhang, L. Mikelsons. **2024 IEEE 63rd Conference on Decision and Control(CDC)**. *Inverse Modeling with Conditional Normalizing Flows for Data-Driven Model Predictive Control*. doi:10.1109/CDC56724.2024.10886862.

Co-Authorships

M.E. Heringhaus, Y. Zhang, A. Zimmermann, L. Mikelsons. **Sensors 2022**. *Towards Reliable Parameter Extraction in MEMS Final Module Testing Using Bayesian Inference*. doi:10.3390/s22145408.

Bibliography

Ryan P. Adams, Jeffrey Pennington, Matthew J. Johnson, Jamie Smith, Yaniv Ovadia, Brian Patton, and James Saunderson. Estimating the spectral density of large implicit matrices, 2018. URL https://arxiv.org/abs/1802.03451.

Edward Allen. *Modeling with itô stochastic differential equations*, volume 22. Springer, 2007.

Govinda Anantha Padmanabha and Nicholas Zabaras. Solving inverse problems using conditional invertible neural networks. *Journal of Computational Physics*, 433:110194, 2021. ISSN 0021-9991. doi: https://doi.org/10.1016/j.jcp.2021.110194. URL https://www.sciencedirect.com/science/article/pii/S0021999121000899.

Lynton Ardizzone, Jakob Kruse, Sebastian Wirkert, Daniel Rahner, Eric W. Pellegrini, Ralf S. Klessen, Lena Maier-Hein, Carsten Rother, and Ullrich Köthe. Analyzing inverse problems with invertible neural networks, 2019a. URL https://arxiv.org/abs/1808.04730.

Lynton Ardizzone, Carsten Lüth, Jakob Kruse, Carsten Rother, and Ullrich Köthe. Guided image generation with conditional invertible neural networks, 2019b. URL https://arxiv.org/abs/1907.02392.

Andrea Asperti. Variance loss in variational autoencoders, 2020. URL https://arxiv.org/abs/2002.09860.

Jens Behrmann, Will Grathwohl, Ricky T. Q. Chen, David Duvenaud, and Jörn-Henrik Jacobsen. Invertible residual networks, 2019. URL https://arxiv.org/abs/1811.00995.

Julian Besag. Markov chain monte carlo methods for statistical inference. *Department of Statistics University of Washington, Seattle, USA*, 2004.

Christopher M. Bishop. *Pattern Recognition and Machine Learning (Information Science and Statistics)*. Springer-Verlag, Berlin, Heidelberg, 2006. ISBN 0387310738.

L. Bittner. L. s. pontryagin, v. g. boltyanskii, r. v. gamkrelidze, e. f. mishechenko, the mathematical theory of optimal processes. viii + 360 s. new york/london 1962. john wiley & sons. preis 90/–. *ZAMM - Journal of Applied Mathematics and Mechanics / Zeitschrift für Angewandte Mathematik und Mechanik*, 43(10–11):514–515, January 1963. ISSN 1521-4001. doi: 10.1002/zamm.19630431023. URL http://dx.doi.org/10.1002/zamm.19630431023.

David M. Blei, Alp Kucukelbir, and Jon D. McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, April 2017. ISSN 1537-274X. doi: 10.1080/01621459.2017.1285773. URL http://dx.doi.org/10.1080/01621459.2017.1285773.

Charles Blundell, Julien Cornebise, Koray Kavukcuoglu, and Daan Wierstra. Weight uncertainty in neural networks, 2015. URL https://arxiv.org/abs/1505.05424.

Léon Bottou. *On-line learning and stochastic approximations*, page 9–42. Cambridge University Press, USA, 1999. ISBN 0521652634.

Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew Dai, Rafal Jozefowicz, and Samy Bengio. Generating sentences from a continuous space. In Stefan Riezler and Yoav Goldberg, editors, *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*, pages 10–21, Berlin, Germany, August 2016. Association for Computational Linguistics. doi: 10.18653/v1/K16-1002. URL https://aclanthology.org/K16-1002/.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations, 2019. URL https://arxiv.org/abs/1806.07366.

Ricky T. Q. Chen, Jens Behrmann, David Duvenaud, and Jörn-Henrik Jacobsen. Residual flows for invertible generative modeling, 2020. URL https://arxiv.org/abs/1906.02735.

Andreas C. Damianou and Neil D. Lawrence. Deep gaussian processes, 2013. URL https://arxiv.org/abs/1211.0358.

Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation, 2015. URL https://arxiv.org/abs/1410.8516.

Shian Du, Yihong Luo, Wei Chen, Jian Xu, and Delu Zeng. To-flow: Efficient continuous normalizing flows with temporal optimization adjoint with moving speed, 2022. URL https://arxiv.org/abs/2203.10335.

Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987. ISSN 0370-2693. doi: https://doi.org/10.1016/0370-2693(87)91197-X. URL https://www.sciencedirect.com/science/article/pii/037026938791197X.

Richard Durrett. *Probability: theory and examples*. Duxbury Press, Belmont, CA, second edition, 1996. ISBN 0-534-24318-5.

Andrew Y. K. Foong, Yingzhen Li, José Miguel Hernández-Lobato, and Richard E. Turner. 'in-between' uncertainty in bayesian neural networks, 2019. URL https://arxiv.org/abs/1906.11537.

Andrew Y. K. Foong, David R. Burt, Yingzhen Li, and Richard E. Turner. On the expressiveness of approximate inference in bayesian neural networks, 2020. URL https://arxiv.org/abs/1909.00719.

Meire Fortunato, Charles Blundell, and Oriol Vinyals. Bayesian recurrent neural networks, 2019. URL https://arxiv.org/abs/1704.02798.

F. Gabbiani and S.J. Cox. *Mathematics for Neuroscientists*. Elsevier science & technology books. Elsevier Science, 2010.

M. Gavrilović, R. Petrović, and D. Šiljak. Adjoint method in the sensitivity analysis of optimal systems. *Journal of the Franklin Institute*, 276(1):26–38, 1963. ISSN 0016-0032. doi: https://doi.org/10.1016/0016-0032(63)90307-7. URL https://www.sciencedirect.com/science/article/pii/0016003263903077.

Andrew Gelman, John B. Carlin, Hal S. Stern, David B. Dunson, Akti Vehtari, and Donald B. Rubin. *Bayesian Data Analysis*. Chapman & Hall/CRC Texts in Statistical Science Series. CRC, Boca Raton, Florida, third edition, 2013. ISBN 9781439840955 1439840954. URL https://stat.columbia.edu/~gelman/book/.

Samuel Gershman and Noah Goodman. Amortized inference in probabilistic reasoning. In *Proceedings of the annual meeting of the cognitive science society*, volume 36, 2014.

Laurent Girin, Simon Leglaive, Xiaoyu Bie, Julien Diard, Thomas Hueber, and Xavier Alameda-Pineda. Dynamical variational autoencoders: A comprehensive review. *Foundations and Trends® in Machine Learning*, 15(1–2):1–175, 2021. ISSN 1935-8245. doi: 10.1561/2200000089. URL http://dx.doi.org/10.1561/2200000089.

Ethan Goan and Clinton Fookes. *Bayesian Neural Networks: An Introduction and Survey*, page 45–87. Springer International Publishing, 2020. ISBN 9783030425531. doi: 10.1007/978-3-030-42553-1_3. URL http://dx.doi.org/10.1007/978-3-030-42553-1_3.

Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models, 2018. URL https://arxiv.org/abs/1810.01367.

Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, December 2017. ISSN 1361-6420. doi: 10.1088/1361-6420/aa9a90. URL http://dx.doi.org/10.1088/1361-6420/aa9a90.

W. K. Hastings. Monte carlo sampling methods using markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970. ISSN 0006-3444. doi: 10.1093/biomet/57.1.97. URL https://doi.org/10.1093/biomet/57.1.97.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL https://arxiv.org/abs/1512.03385.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL https://arxiv.org/abs/2006.11239.

M.F. Hutchinson. A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19(2): 433–450, 1990. doi: 10.1080/03610919008812866. URL https://doi.org/10.1080/03610919008812866.

Pavel Izmailov, Sharad Vikram, Matthew D. Hoffman, and Andrew Gordon Wilson. What are bayesian neural network posteriors really like?, 2021. URL https://arxiv.org/abs/2104.14421.

Michael I. Jordan, Zoubin Ghahramani, Tommi S. Jaakkola, and Lawrence K. Saul. *An Introduction to Variational Methods for Graphical Models*. Springer Netherlands, Dordrecht, 1998. ISBN 978-94-011-5014-9. doi: 10.1007/978-94-011-5014-9_5. URL https://doi.org/10.1007/978-94-011-5014-9_5.

Grigory Khromov and Sidak Pal Singh. Some fundamental aspects about lipschitz continuity of neural networks, 2024. URL https://arxiv.org/abs/2302.10886.

Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions, 2018. URL https://arxiv.org/abs/1807.03039.

Diederik P. Kingma and Max Welling. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019. ISSN 1935-8245. doi: 10.1561/2200000056. URL http://dx.doi.org/10.1561/2200000056.

Diederik P Kingma and Max Welling. Auto-encoding variational bayes, 2022. URL https://arxiv.org/abs/1312.6114.

Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing flows: An introduction and review of current methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, November 2021. ISSN 1939-3539. doi: 10.1109/tpami.2020.2992934. URL http://dx.doi.org/10.1109/TPAMI.2020.2992934.

Jakob Kruse, Gianluca Detommaso, Ullrich Köthe, and Robert Scheichl. Hint: Hierarchical invertible neural transport for density estimation and bayesian inference, 2021. URL https://arxiv.org/abs/1905.10687.

S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 00034851. URL http://www.jstor.org/stable/2236703.

Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations, 2020. URL https://arxiv.org/abs/1710.10121.

David J.C MacKay. Bayesian neural networks and density networks. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 354(1):73–80, 1995. ISSN 0168-9002. doi: https://doi.org/10.1016/0168-9002(94)00931-7. URL https://www.sciencedirect.com/science/article/pii/0168900294009317. Proceedings of the Third Workshop on Neutron Scattering Data Analysis.

Charles C. Margossian and David M. Blei. Amortized variational inference: When and why?, 2024. URL https://arxiv.org/abs/2307.11018.

Jorge Martínez-Palomera, Joshua S. Bloom, and Ellianna S. Abrahams. Deep generative modeling of periodic variable stars using physical parameters, 2020. URL https://arxiv.org/abs/2005.07773.

Arya D. McCarthy, Xian Li, Jiatao Gu, and Ning Dong. Addressing posterior collapse with mutual information for improved variational neural machine translation. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8512–8525, Online, July

2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.753. URL https://aclanthology.org/2020.acl-main.753/.

Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 06 1953. ISSN 0021-9606. doi: 10.1063/1.1699114. URL https://doi.org/10.1063/1.1699114.

Ibomoiye Domor Mienye and Theo G. Swart. Deep autoencoder neural networks: A comprehensive review and new perspectives. *Archives of Computational Methods in Engineering*, Mar 2025. ISSN 1886-1784. doi: 10.1007/s11831-025-10260-5. URL https://doi.org/10.1007/s11831-025-10260-5.

Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL http://probml.github.io/book1.

Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023. URL http://probml.github.io/book2.

Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do bayesian inference, 2024. URL https://arxiv.org/abs/2112.10510.

Radford M. Neal. *Priors for Infinite Networks*, pages 29–53. Springer New York, New York, NY, 1996. ISBN 978-1-4612-0745-0. doi: 10.1007/978-1-4612-0745-0_2. URL https://doi.org/10.1007/978-1-4612-0745-0_2.

Jose L. Paniagua and Jesús A. López. Nonlinear system identification using modified variational autoencoders. *Intelligent Systems with Applications*, 22:200344, 2024. ISSN 2667-3053. doi: https://doi.org/10.1016/j.iswa.2024.200344. URL https://www.sciencedirect.com/science/article/pii/S2667305324000206.

George Papamakarios, Theo Pavlakou, and Iain Murray. Masked autoregressive flow for density estimation, 2018. URL https://arxiv.org/abs/1705.07057.

George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference, 2021. URL https://arxiv.org/abs/1912.02762.

Stefan Radev. Deep learning architectures for amortized bayesian inference in cognitive modeling, 2021. URL https://archiv.ub.uni-heidelberg.de/volltextserver/id/eprint/30807.

Stefan T. Radev, Ulf K. Mertens, Andreas Voss, Lynton Ardizzone, and Ullrich Köthe. Bayesflow: Learning complex stochastic models with invertible neural networks, 2020. URL https://arxiv.org/abs/2003.06281.

Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian processes for machine learning*. MIT Press, 2008.

Danilo Jimenez Rezende and Shakir Mohamed. Variational inference with normalizing flows, 2016. URL https://arxiv.org/abs/1505.05770.

Janosh Riebesell, Kapil Sachdeva, Juan Emmanuel Johnson, François Rozet, AifuHan, Byeongkeun Ahn, He Wang, Hossein Pourbozorg, Hyounguk Shon, John Franklin Crenshaw, Kristian Hartikainen, Kyle Cranmer, Matthew Skiffington, Maxime Vandegar, Rafael Orozco, Sandeep Nagar, Thorsten Glüsenkamp, Vincent Stimper, and kleinicke. janosh/awesome-normalizing-flows: v1.0.0, 2023.

Lars Ruthotto and Eldad Haber. Deep neural networks motivated by partial differential equations, 2018. URL https://arxiv.org/abs/1804.04272.

J. Schwartz. The formula for change in variables in a multiple integral. *The American Mathematical Monthly*, 61(2):81–85, 1954. ISSN 00029890, 19300972. URL http://www.jstor.org/stable/2307790.

Kumar Shridhar, Felix Laumann, and Marcus Liwicki. A comprehensive guide to bayesian convolutional neural network with variational inference, 2019. URL https://arxiv.org/abs/1901.02731.

Daniele Silvestro and Tobias Andermann. Prior choice affects ability of bayesian neural networks to identify unknowns, 2020. URL https://arxiv.org/abs/2005.04987.

John Skilling. *The Eigenvalues of Mega-dimensional Matrices*, pages 455–466. Springer Netherlands, Dordrecht, 1989. ISBN 978-94-015-7860-8. doi: 10.1007/978-94-015-7860-8_48. URL https://doi.org/10.1007/978-94-015-7860-8_48.

Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf.

Simo Särkkä. The use of gaussian processes in system identification, 2019. URL https://arxiv.org/abs/1907.06066.

Martin J Wainwright and Michael I Jordan. *Graphical Models, Exponential Families, and Variational Inference*. Now Publishers Inc., Hanover, MA, USA, 2008. ISBN 1601981848.

Yixin Wang, David M. Blei, and John P. Cunningham. Posterior collapse and latent variable non-identifiability, 2023. URL https://arxiv.org/abs/2301.00537.

Christina Winkler, Daniel Worrall, Emiel Hoogeboom, and Max Welling. Learning likelihoods with conditional normalizing flows, 2023. URL https://arxiv.org/abs/1912.00042.



YI ZHANG

PhD in Applied Computer Science

Augsburg | 015236410279 | zhangyi1353173@yahoo.com
https://www.linkedin.com/in/yi-zhang-2020/

Soon-to-graduate PhD specializing in probabilistic deep learning for real-world industrial applications. Experienced in bridging state-of-the-art AI research with practical deployment, with a focus on developing reliable, robust, and efficient system identification and state estimation methods – essential for system observation, control, and decision making

Education

University of Augsburg 04.2021-03.2026 (estimated)
Doctor of Engineering in Applied Computer Science
Title of Dissertation: Probabilistic System Identification and State Estimation for Dynamical Systems via Normalizing Flows
https://www.uni-augsburg.de/de/fakultaet/fai/informatik/prof/imech/team/yi-zhang/

University of Stuttgart 10.2017 – 09.2020
Master of Science in Electrical and Electronics Engineering
Major: Signal processing, Machine Learning
Master Thesis: Feature Selection using Attention Mechanism in Deep Learning
Grade: 1.6

TH Aschaffenburg University of Applied Science 09.2016 – 09.2017
Bachelor of Engineering in Mechatronics (Exchange Program)
Major: Drive technology and robotics, Application of micro-electrotechnology
Bachelor Thesis: Algorithms for avoiding singularity in industrial robots
Grade: 1.5

University of Tongji, Shanghai 09.2013 – 09.2016
Bachelor of Engineering in Mechatronics
Grade: 1.8

Experience

Trumpf Werkzeugmaschinen GmbH, Ditzingen 05.2019 – 10.2019
Intern in R&D Pre-development Production Platform MT
- Support in the development of a prediction model for process-reliable part removal on a fully automated laser cutting machine
- Further development and implementation of a novel computer vision approach for teaching a finite element structure simulation through U-Net architectures, with Keras
- Evaluation and training of the models as well as realization and identification of improvement potentials to increase model accuracy

Publications

Zhang, Y. and Mikelsons, L., IMFlow: Inverse Modeling with Conditional Normalizing Flows for Data-Driven Model Predictive Control, 2024 IEEE 63rd Conference on Decision and Control (CDC), Milan, Italy, 2024, pp. 3353-3359, doi: 10.1109/CDC56724.2024.10886862.
Zhang, Y. and Mikelsons, L., Solving Stochastic Inverse Problems with Stochastic BayesFlow, 2023 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM), Seattle, WA, USA, 2023, pp. 966-972, doi: 10.1109/AIM46323.2023.10196190.
Zhang, Y. and Mikelsons, L., Sensitivity-guided iterative parameter identification and data generation with BayesFlow and PELS-VAE for model calibration. Adv. Model. and Simul. in Eng. Sci. 10, 9 (2023). doi: 10.1186/s40323-023-00246-y.
Heringhaus, M.E.; Zhang, Y.; Zimmermann, A.; Mikelsons, L., Towards Reliable Parameter Extraction in MEMS Final Module Testing Using Bayesian Inference. Sensors 2022, 22, 5408. doi: 10.3390/s22145408

Programming Languages

Python, Julia
MATLAB/Simulink, C++

Languages

English: Professional Proficiency
German: Professional Proficiency
Chinese: Native Proficiency

Technical Skills

Machine Learning/Deep Learning
Reinforcement Learning
Mathematics & Statistics
Data processing & -analysis
ML-Frameworks: PyTorch, Jax, TensorFlow/Keras

Problem-solving expertise

Implementation and optimization of algorithms
Interdisciplinary work
Creativity & critical thinking
Ability to think abstractly

Soft Skills

Explanation of complex concepts
Collaboration with researchers and industry partners
Independence & personal responsibility
Time management & prioritisation
Teamwork & collaboration
Openness to feedback
Curiosity & lifelong learning
Perseverance

Third-party funded projects

PHyMoS 04.2021 – 02.2024
(https://phymos.de/)
OpenScaling 11.2023 – 10.2026
(https://openscaling.org/)

Appendix

Acknowledgement

Eidesstattliche Versicherung

Statutory Declaration

1. Bei der eingereichten Dissertation zu dem Thema ***Probabilistic System Identification and State Estimation via Normalizing Flows*** handelt es sich um meine eigenständig erbrachte Leistung.
   *I herewith formally declare that I have written the submitted dissertation **Probabilistic System Identification and State Estimation via Normalizing Flows** independently.*
2. Ich habe nur die angegebenen Quellen und Hilfsmittel benutzt und mich keiner unzulässigen Hilfe Dritter bedient. Insbesondere habe ich wörtlich oder sinngemäß aus anderen Werken übernommene Inhalte als solche kenntlich gemacht.
   I did not use any third party support except for the quoted literature and other sources mentioned in the text. Content from other work, either literally or in content, has been declared as such.
3. Die Arbeit oder Teile davon habe ich bislang nicht an einer Hochschule des In- oder Auslands als Bestandteil einer Prüfungs- oder Qualifikationsleistung vorgelegt.
   The thesis has not been submitted to any examination body in this, or similar, form.
4. Die Richtigkeit der vorstehenden Erklärungen bestätige ich.
   I confirm the correctness of the aforementioned declarations.
5. Die Bedeutung der eidesstattlichen Versicherung und die strafrechtlichen Folgen einer unrichtigen oder unvollständigen eidesstattlichen Versicherung sind mir bekannt. Ich versichere an Eides statt, dass ich nach bestem Wissen die reine Wahrheit erklärt und nichts verschwiegen habe.
   I am aware of the legal consequences of this declaration. To the best of my knowledge I have told the pure truth and not concealed anything.

Augsburg, 16.09.2025 _____

Angabe zu verwendeten KI-basierter Elektronischer Hilfsmittel

Information on the use of AI-based tools

Zur Dokumentation der verwendeten Hilfsmittel ist der schriftlichen Ausarbeitung ein besonderer Anhang hinzugefügt, der eine Liste und Beschreibung aller verwendeter KI-basierter Hilfsmittel enthält. Der besondere Anhang zur Dokumentation der verwendeten Hilfsmittel erfüllt folgende Kriterien:

To document the tools used, a special appendix has been added to the written report, which contains a list and description of all AI-based tools used. The special appendix to document the tools used meets the following criteria:

1. Auflistung der Ziele, für die die KI-basierten Hilfsmittel in der vorliegenden Arbeit eingesetzt wurden.
   List of the goals for which the AI-based tools were used in this work.

2. Dokumentation der Verwendungsweise der KI-basierten Hilfsmittel.
   Documentation of how AI-based tools are used.

3. Nennung der Kapitel und Abschnitte der vorliegenden Arbeit, in denen die KI-basierten Hilfsmittel eingesetzt wurden, um Inhalte zu erzeugen.
   Identification of the chapters and sections of this work in which AI-based tools were used to generate content.

Der Gebrauch dieser Hilfsmittel inklusive Art, Ziel und Umfang des Gebrauchs wurde mit meinem offiziellen Betreuer **Prof.Dr.Ing. Lars Mikelsons** abgesprochen.

*The use of these aids, including the type, purpose and extent of use, has been agreed with my official supervisor **Prof.Dr.Eng. Lars Mikelsons**.*

Mir ist bewusst, dass insbesondere der Versuch einer nicht dokumentierten Nutzung KI- basierter Hilfsmittel als Täuschungsversuch zu werten ist:

I am aware that in particular the attempt to use AI-based tools without documentation is to be regarded as an attempt at deception:

Gem. § 16 Abs. 2 der Promotionsordnung „Dr. med. / dent.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Kandidat/die Kandidatin bei einer Promotionsleistung getäuscht hat, so können einzelne oder alle Promotionsleistungen für ungültig erklärt werden. In schweren Fällen kann die Zulassung zum Promotionsverfahren zurückgenommen werden.“

According to § 16 paragraph 2 of the doctoral regulations for "Dr. med. / dent.": "If it emerges before the doctoral certificate is issued that the candidate has cheated in a doctoral achievement, individual or all doctoral achievements can be declared invalid. In serious cases, admission to the doctoral process can be withdrawn.

Und § 16 Abs. 2 der Promotionsordnung „Dr. sc. hum.“: „Ergibt sich vor Aushändigung der Promotionsurkunde, dass der Doktorand / Doktorandin bei einer Promotionsleistung getäuscht hat, so kann der Promotionsausschuss diese Promotionsleistung oder alle bisher erbrachten Promotionsleistungen für ungültig erklären. In besonders schweren Fällen kann der Promotionsausschuss die Annahme als Doktorand / Doktorandin endgültig widerrufen.“

And § 16 paragraph 2 of the doctoral regulations “Dr. sc. hum.”: “If it emerges before the doctoral certificate is issued that the doctoral candidate has cheated in a doctoral achievement, the doctoral committee can declare this doctoral achievement or all doctoral achievements to date invalid. In particularly serious cases, the doctoral committee can permanently revoke the acceptance as a doctoral candidate.”

Augsburg, 16.09.2025 _____